



Spécification technique du projet S301

Groupe B

2025-2026

Table des matières

1. Répartition des tâches	2
2. Architecture du code	2
3. Organisation des données	3
3.1. Organisation des données du côté serveur (Langage C)	3
3.1.1. Variables globales	3
3.1.2. Structure de données	3
3.2. Organisation des données du côté client	4
3.3. Organisation des données dans le base de données	5
4. Diagrammes de séquences	6
4.1. Diagramme de séquence de la commande "Créer_carte"	6
4.2. Diagramme de séquence de la commande "Échanger_carte"	7
4.3. Diagramme de séquence de la commande "Combattre_carte"	7
5. Protocole applicatif	8
5.1. Format général des messages	8
5.2. Types de messages	8
5.2.1. Connexion / Déconnexion	8
5.2.2. Création de carte	9
5.2.3. Echange de cartes	9
5.2.3. Combat de cartes	9

1. Répartition des tâches

Membres	Rôles
Bryan Lachal	Codeur, testeur
Khadija Zouhair	Codeur, qualitiicienne
Louis Bedetti	Chef de projet, codeur, qualitiicien, concepteur
Léo Coletta	Codeur, testeur, concepteur

2. Architecture du code

Nous devons définir les grands choix techniques effectués, d'abord au niveau du serveur.

Le premier de ces choix concerne l'utilisation du multi-process ou du multi-thread. Il faut en effet faire en sorte que le serveur puisse effectuer plusieurs opérations simultanément sans bloquer. Par exemple, si plusieurs joueurs sont connectés et essaient d'échanger des cartes entre eux, il ne faut pas que le serveur traite chaque demande une après l'autre. Dans le cas du multi-process, le serveur crée plusieurs processus qui traitent chaque client. Cela a l'avantage de séparer les ressources mais est assez complexe à mettre en place. Du côté du multi-thread, il n'y a qu'un seul processus, mais chaque client est géré par un thread, ce qui est donc plus facile à développer mais tous les threads partagent les mêmes ressources. Cela veut dire qu'un bug dans un seul thread peut planter tout le serveur. De plus, il faut pouvoir gérer la synchronisation, c'est-à-dire savoir quoi faire si deux joueurs modifient la même information en même temps. Malgré tout, nous avons opté pour le multi-thread car c'est un procédé plus facilement implémentable dans un projet de cette taille.

Dans cette application, le serveur et les clients communiquent avec des sockets. Il y a plusieurs choix conceptuels à faire pour ceux-ci. D'abord, nous avons opté pour des socket TCP, car ce protocole est adapté à ce type d'application de par sa fiabilité. Ensuite, nous avons choisi un mode de communication bloquant : tant qu'il n'y a pas de message, la lecture du socket bloque, ce qui évite d'avoir une boucle de gestion. La communication entre les sockets se fera par des messages suivant un protocole que nous avons défini (voir section 5 : Protocole applicatif).

Au niveau client, nous avons d'abord fait le choix d'une interface épurée, simple à comprendre pour l'utilisateur.

Le client reconnaît plusieurs événements : certains déclenchés par l'utilisateur, d'autres par le serveur. Ces événements vont à leur tour déclencher différentes fonctions de traitements de ces événements, comme des affichages spécifiques à l'écran ou des mises à jour de l'état du joueur/de ses cartes.

Événements utilisateurs : création de carte, demande d'échange, proposition de combat, déconnexion.

Événements serveur : confirmation de création de carte, notifications d'échange, notifications de combat.

Le programme côté client utilise un modèle orienté objet. Il sera donc découpé en plusieurs classes, qui sont détaillées dans la section Organisation des données (3.2).

Enfin, l'application utilise une base de données sous PostgreSQL qui sauvegarde en réalité une blockchain contenant toutes les informations enregistrées : les joueurs, les cartes qu'ils possèdent, les combats qu'ils ont fait, etc. Le schéma relationnel de la BDD est détaillé dans la section Organisation des données (3.3).

3. Organisation des données

L'organisation des données est différente et est décrite séparément pour le côté serveur, le côté client et la BDD.

Elle définit la façon dont les informations sont manipulées et stockées dans le système.

3.1. Organisation des données du côté serveur (Langage C)

Le serveur étant développé en langage C, il est responsable de la gestion des joueurs connectés, des cartes créées par les joueurs, ainsi que la blockchain.

3.1.1. Variables globales

Certaines constantes et paramètres sont définis globalement, par exemple le nombre max de joueurs connectés sur le serveur (MaxClients), le nombre max de cartes possédées par joueur (MaxCardPerClient), la difficulté du minage de la blockchain, ainsi que la socket principale d'écoute du serveur.

3.1.2. Structure de données

Plusieurs types de structure de données sont utilisées afin d'organiser les données :

Structure Client : int ID_Client, char Username, int Socket, int NbCards, struct Card *main.

Structure Card : int ID_Card, char Name, int Attack, int Defense, int MaxHP, int CurrentHP.

Structure Bloc : int ID_Bloc, long timestamp, char data_action, char PreviousHash, char Hash, struct Bloc *next.

Structure Blockchain : Bloc *genesis, Bloc *previous, int Size.

Le serveur garde donc en mémoire la liste de tous les joueurs connectés à ce dernier, leur liste de cartes et la blockchain courante.

3.2. Organisation des données du côté client

Les clients, développés en Java avec JavaFx, sont orientés objet et intègrent une interface graphique. Ils gèrent donc les actions des joueurs et la communication avec le serveur.

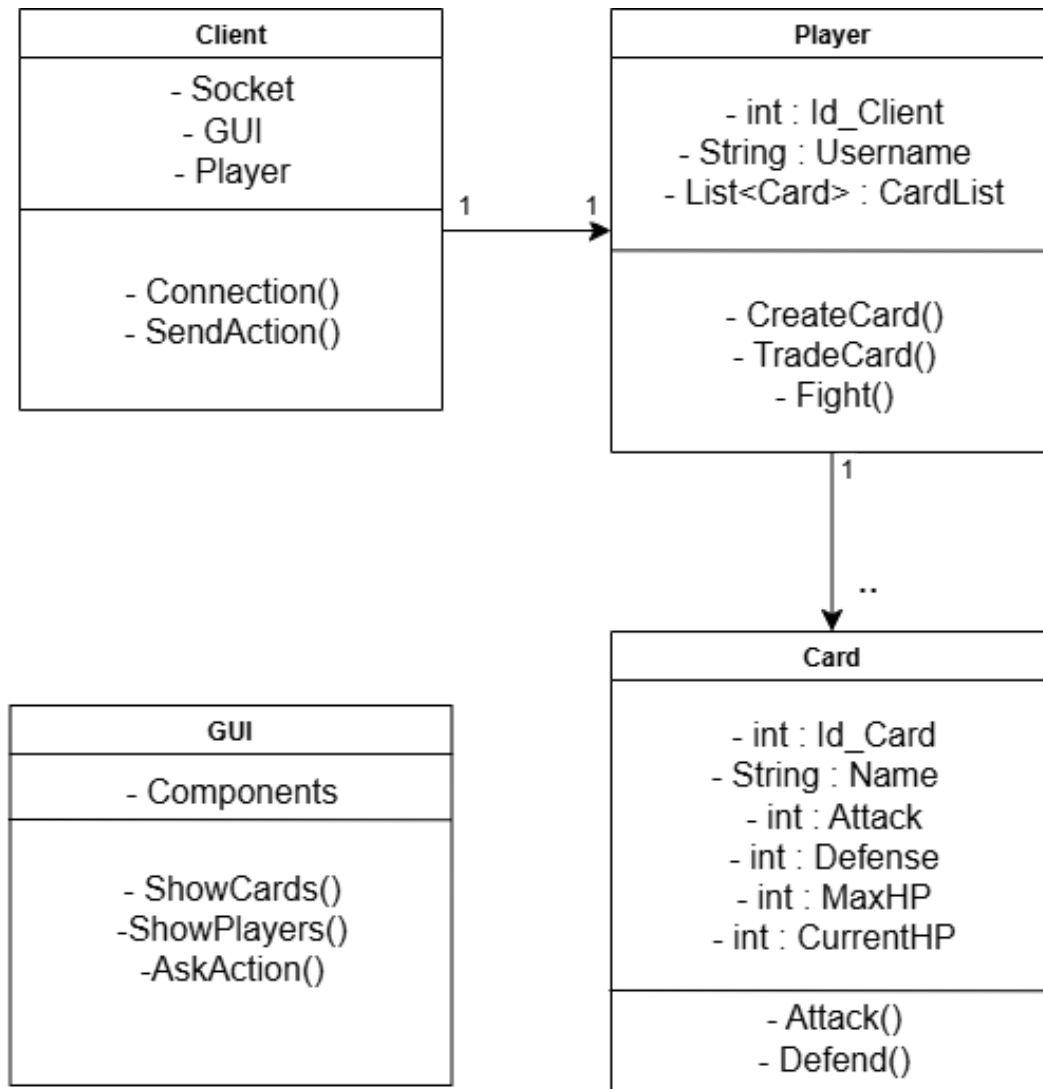


Diagramme de classe de l'application côté client

3.3. Organisation des données dans le base de données

La base de données, gérée avec PostgreSQL, s'occupe de sauvegarder et de restaurer le contexte du serveur. Elle conserve principalement la blockchain et les informations relatives aux joueurs et aux cartes.

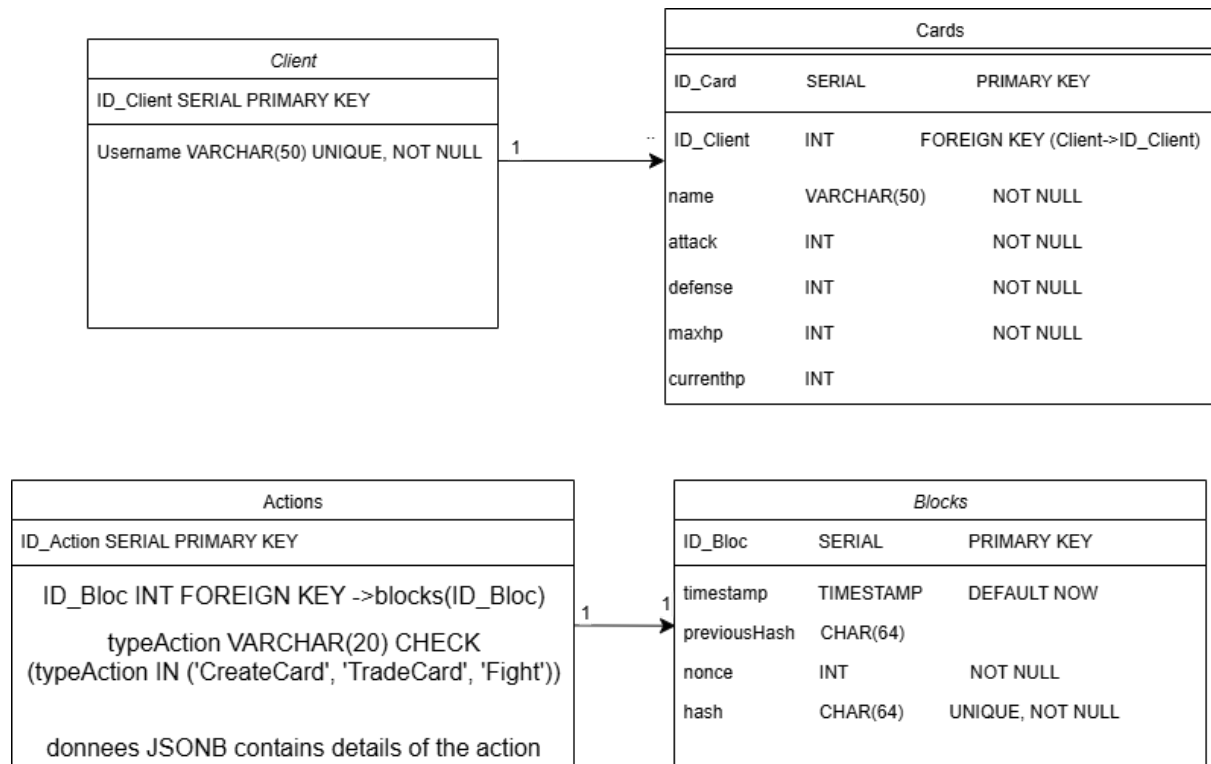
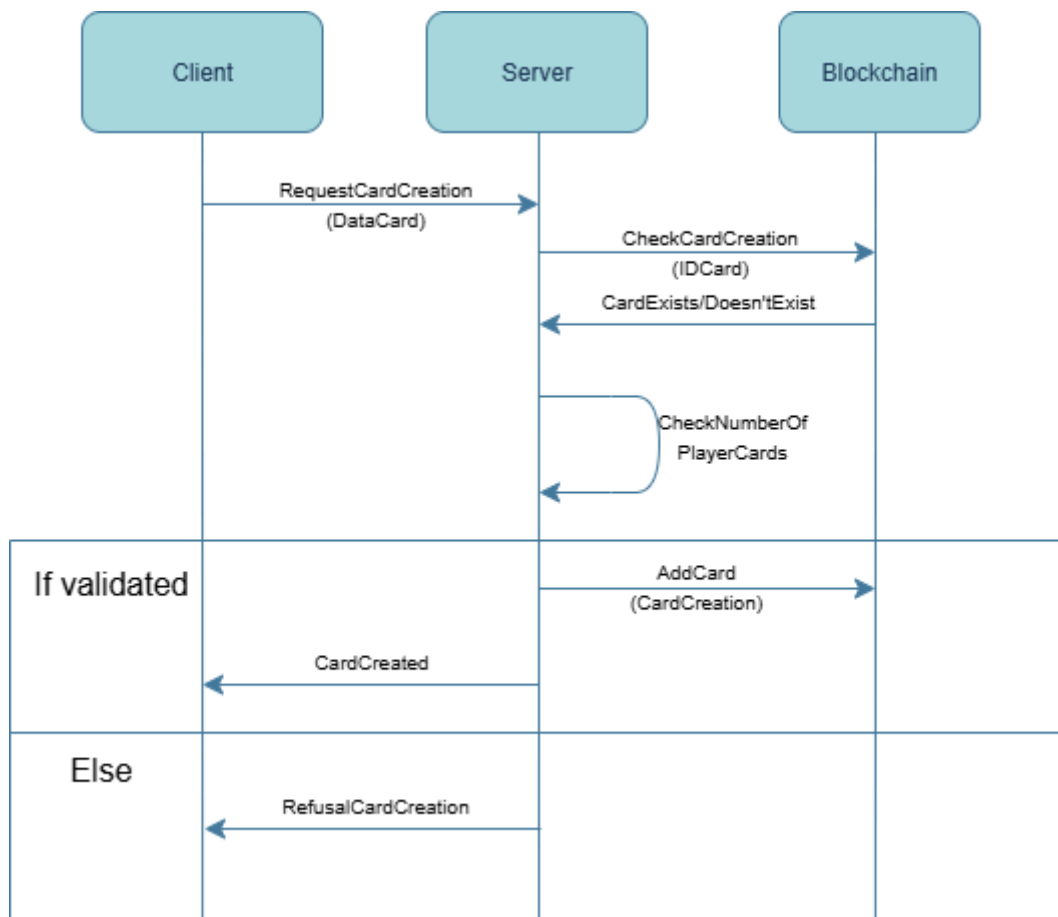


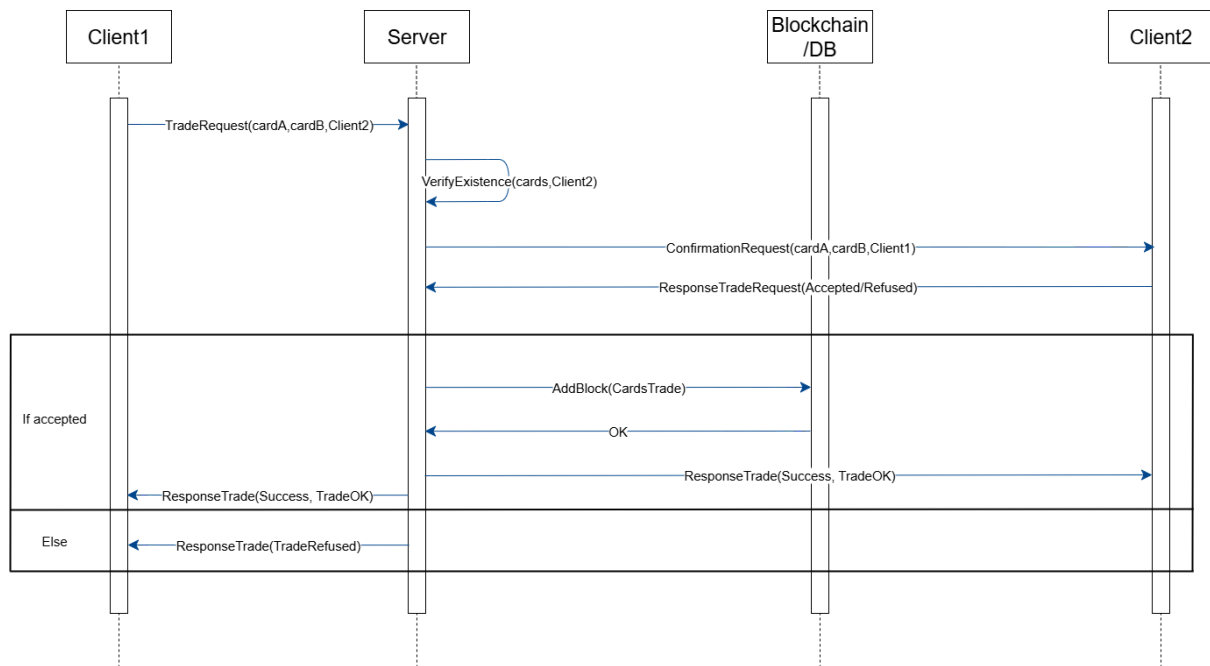
Schéma relationnel de la base de données

4. Diagrammes de séquences

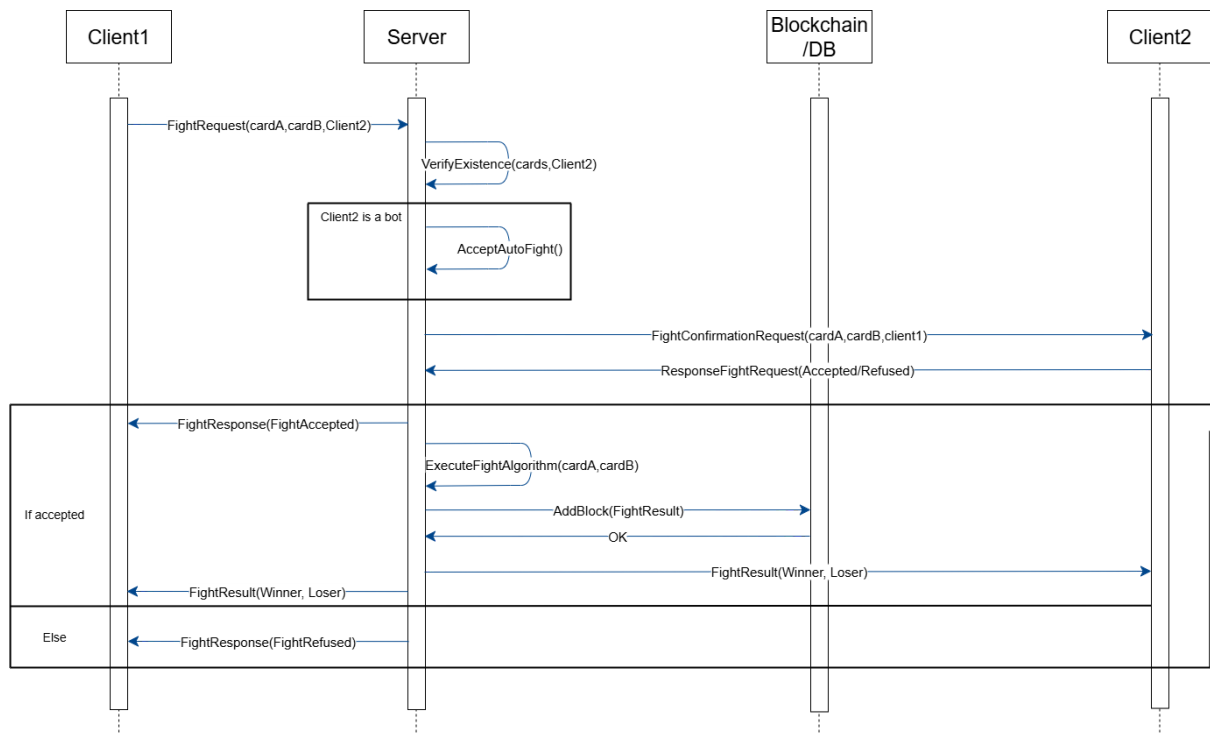
4.1. Diagramme de séquence de la commande “Créer_carte”



4.2. Diagramme de séquence de la commande “Échanger_carte”



4.3. Diagramme de séquence de la commande “Combattre_carte”



5. Protocole applicatif

5.1. Format général des messages

- Transport : TCP
- Encodage : JSON
- Un message contient au minimum :
 - Un type : request, response ou notification
 - Le nom de l'action
 - Des données (data)
- Les identifiants de joueurs, cartes et actions sont uniques
- Les réponses contiennent un champ status = "OK" ou "ERROR" avec un éventuel message

5.2. Types de messages

5.2.1. Connexion / Déconnexion

- Connexion client → Serveur
type : request
nom : LOGIN
data : id_client
- Réponse serveur
type : response
nom : LOGIN
data : id_client
status : OK ou ERROR
- Déconnexion client → Serveur
type : request
nom : LOGOUT
data : id_client
- Réponse serveur
type : response
nom : LOGOUT
data : id_client
status : OK ou ERROR

5.2.2. Création de carte

- Client → Serveur
type : request
nom : RequestCardCreation
data : nomCarte, pv, attaque
- Serveur → Client
type : response
nom : CardCreated
data : idCarte
status : OK ou ERROR

5.2.3. Echange de cartes

- Client1 → Serveur
type : request
nom : TradeRequest
data : cardA, cardB, Client2
- Serveur → Client2
type : request
nom : ConfirmationRequest
data : cardA, cardB, Client1
status : OK ou ERROR
- Client2 → Serveur
type : response
nom : ResponseTradeRequest
data : Accepted / Refused
- Serveur → Client1
type : response
nom : ResponseTrade
data : TradeOK / TradeRefused
status : OK ou ERROR

5.2.3. Combat de cartes

- Client1 → Serveur
type : request
nom : FightRequest
data : cardA, cardB, Client2

- Serveur → Client2
type : request
nom : FightConfirmationRequest
data : cardA, cardB, Client1
status : OK ou ERROR
- Client2 → Serveur
type : response
nom : ResponseFightRequest
data : Accepted / Refused
- Serveur → Clients
type : response
nom : FightResult
data : Winner, Loser
status : OK ou ERROR
- Serveur → Client1
type : response
nom : FightResponse
data : FightRefused / FightAccepted
status : OK ou ERROR